Shruti Bhoi

backend-engineering-case-study-bynry

PART 1: Code Review & Debugging (FINAL ANSWER)

1. Issues Identified

1. **No Input Validation**
   The code directly accesses data['name'], data['sku'], etc. without checking if they exist or are valid.
2. **SKU Uniqueness Not Enforced**
   There is no check to ensure that the SKU is unique across the platform.
3. **Multiple Database Commits (No Transaction Handling)**
   The product and inventory are committed separately, which can lead to inconsistent data.
4. **Products Cannot Exist in Multiple Warehouses (Design Issue)**
   The product is tied to a single warehouse_id, which violates the requirement.
5. **No Error Handling**
   If any database operation fails, the API will crash without returning a proper error response.
6. **Price Not Validated as Decimal**
   The price field is not validated, which may cause incorrect data storage.
7. **Initial Quantity Not Validated**
   initial_quantity may be missing or invalid (negative or null).
8. **No Handling for Optional Fields**
   Some fields might be optional, but the code assumes all are mandatory.

---

2. Impact of Issues in Production

1. **Invalid Input → Application Crash**
   Missing fields will cause KeyError and crash the API.
2. **Duplicate SKU → Data Inconsistency**
   Multiple products with the same SKU can break business logic and reporting.
3. **Partial Data Creation → Inconsistent State**
   Product may be created but inventory not created if second commit fails.
4. **Incorrect Data Modeling → Scalability Issues**
   Restricting products to one warehouse limits system functionality.
5. **No Error Handling → Poor User Experience**
   Users receive unclear or no response on failure.
6. **Invalid Price → Financial Errors**
   Incorrect pricing can affect billing and analytics.
7. **Invalid Inventory → Wrong Stock Levels**
   Negative or missing quantities can break inventory tracking.

3. Corrected Code

```
from flask import request, jsonify
from sqlalchemy.exc import IntegrityError

@app.route('/api/products', methods=['POST'])
def create_product():
    data = request.get_json()

    required_fields = ['name', 'sku', 'price', 'warehouse_id', 'initial_quantity']
    for field in required_fields:
        if field not in data:
            return jsonify({"error": f"{field} is required"}), 400

    if data['price'] < 0:
        return jsonify({"error": "Price must be a positive value"}), 400

    if data['initial_quantity'] < 0:
        return jsonify({"error": "Quantity cannot be negative"}), 400

    try:
        existing_product = Product.query.filter_by(sku=data['sku']).first()
        if existing_product:
            return jsonify({"error": "SKU already exists"}), 400
        with db.session.begin():
            product = Product(
                name=data['name'],
                sku=data['sku'],
                price=data['price']
            )
            db.session.add(product)
            db.session.flush()  # Get product.id before commit
```

```
        inventory = Inventory(
            product_id=product.id,
            warehouse_id=data['warehouse_id'],
            quantity=data['initial_quantity']
        )
        db.session.add(inventory)

    return jsonify({
        "message": "Product created successfully",
        "product_id": product.id
    }), 201

except IntegrityError:
    db.session.rollback()
    return jsonify({"error": "Database integrity error"}), 500

except Exception as e:
    db.session.rollback()
    return jsonify({"error": str(e)}), 500
```

4. Explanation of Fixes

1. **Validation Added**
   Ensures all required fields exist and values are valid.
2. **SKU Check Implemented**
   Prevents duplicate product entries.
3. **Transaction Handling (Atomic Operation)**
   Both product and inventory are created together or not at all.
4. **Improved Data Model**
   Product is no longer tied directly to a warehouse.
5. **Error Handling Added**
   Returns meaningful error messages instead of crashing.
6. **Flush Before Inventory Insert**
   Ensures product ID is available before creating inventory.

backend-engineering-case-study-bynry

PART 2: Database Design

1. Database Schema Design

The system is designed to support **multi-company, multi-warehouse inventory management with supplier relationships and product bundling**.

1. Companies Table

```
Companies (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

---

2. Warehouses Table

```
Warehouses (
  id INT PRIMARY KEY AUTO_INCREMENT,
  company_id INT NOT NULL,
  name VARCHAR(255) NOT NULL,
  location VARCHAR(255),
  FOREIGN KEY (company_id) REFERENCES Companies(id)
);
```

---

3. Products Table

```
Products (
  id INT PRIMARY KEY AUTO_INCREMENT,
  company_id INT NOT NULL,
  name VARCHAR(255) NOT NULL,
  sku VARCHAR(100) UNIQUE NOT NULL,
  price DECIMAL(10,2) NOT NULL,
  product_type VARCHAR(50), -- normal / bundle
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (company_id) REFERENCES Companies(id)
);
```

4. Inventory Table (Core Table)

```
Inventory (
   id INT PRIMARY KEY AUTO_INCREMENT,
   product_id INT NOT NULL,
   warehouse_id INT NOT NULL,
   quantity INT NOT NULL DEFAULT 0,
   last_updated TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   UNIQUE (product_id, warehouse_id),
   FOREIGN KEY (product_id) REFERENCES Products(id),
   FOREIGN KEY (warehouse_id) REFERENCES Warehouses(id)
);
```

---

5. Inventory History Table (Tracking Changes)

```
Inventory_History (
   id INT PRIMARY KEY AUTO_INCREMENT,
   product_id INT,
   warehouse_id INT,
   change_type VARCHAR(50), -- add / remove / update
   quantity_changed INT,
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
   FOREIGN KEY (product_id) REFERENCES Products(id),
   FOREIGN KEY (warehouse_id) REFERENCES Warehouses(id)
);
```

---

6. Suppliers Table

```
Suppliers (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(255) NOT NULL,
   contact_email VARCHAR(255),
   phone VARCHAR(20)
);
```

7. Product_Suppliers Table (Many-to-Many)

```
Product_Suppliers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    supplier_id INT,
    PRIMARY KEY (product_id, supplier_id),
    FOREIGN KEY (product_id) REFERENCES Products(id),
    FOREIGN KEY (supplier_id) REFERENCES Suppliers(id)
);
```

---

8. Product Bundles Table

```
Product_Bundles (
    id INT PRIMARY KEY AUTO_INCREMENT,
    bundle_product_id INT,
    child_product_id INT,
    quantity INT NOT NULL,
    FOREIGN KEY (bundle_product_id) REFERENCES Products(id),
    FOREIGN KEY (child_product_id) REFERENCES Products(id)
);
```

---

9. Sales Table (For Low Stock Logic)

```
Sales (
    id INT PRIMARY KEY AUTO_INCREMENT,
    product_id INT,
    warehouse_id INT,
    quantity_sold INT,
    sale_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (product_id) REFERENCES Products(id),
    FOREIGN KEY (warehouse_id) REFERENCES Warehouses(id)
);
```

---

backend-engineering-case-study-bynry

2. Missing Requirements (Questions to Product Team)

1. Can a product have **multiple suppliers**, or only one preferred supplier?
2. How is **low stock threshold defined** (per product, per warehouse, or globally)?
3. What defines **"recent sales activity"** (last 7 days, 30 days, etc.)?
4. Can **bundles dynamically change**, or are they fixed once created?
5. Should inventory updates be **real-time or batch processed**?
6. Can warehouses **transfer stock between each other**?
7. Should we support **backorders when stock is zero**?
8. Are suppliers linked at **company level or globally across platform**?

---

3. Design Decisions & Justification

1. Separate Inventory Table

- Allows **products to exist in multiple warehouses**
- Supports **different stock levels per location**

---

2. Unique SKU Constraint

- Ensures **data consistency across platform**
- Prevents duplicate product entries

---

3. Inventory History Table

- Enables **tracking of stock changes**
- Useful for **auditing and analytics**

---

4. Many-to-Many Product-Supplier Mapping

- A product can have **multiple suppliers**
- Improves **flexibility in procurement**

---

5. Bundle Design (Self-Referencing Table)

- Supports **composite products**
- Enables **nested product relationships**

---

6. Indexing Strategy

- Index on sku → Fast product lookup
- Index on (product_id, warehouse_id) → Fast inventory queries
- Index on sale_date → Efficient recent sales filtering

---

7. Scalability Considerations

- Normalized schema reduces redundancy
- Supports **multi-tenant architecture (company-based)**
- Designed for **high read/write operations**

This database design:

- Supports **multi-warehouse inventory**
- Handles **real-time stock tracking**
- Enables **supplier relationships**
- Supports **bundled products**
- Scales for **multiple companies**

backend-engineering-case-study-bynry

PART 3: API Implementation (Low Stock Alerts)

1. Assumptions

Since some requirements were not fully defined, the following assumptions are made:

1. **Low stock threshold** is stored per product.
2. **Recent sales activity** means sales within the last **30 days**.
3. Each product has at least one **primary supplier**.
4. **Days until stockout** is calculated using average daily sales.
5. A company can have **multiple warehouses**.

---

2. API Endpoint

GET /api/companies/{company_id}/alerts/low-stock

---

3. Implementation (Flask + SQLAlchemy)

```
from flask import jsonify
from datetime import datetime, timedelta
from sqlalchemy.sql import func

@app.route('/api/companies/<int:company_id>/alerts/low-stock', methods=['GET'])
def get_low_stock_alerts(company_id):

    try:
        alerts = []
        recent_days = 30
        recent_date = datetime.utcnow() - timedelta(days=recent_days)

        # Step 1: Get all products for the company
        products = Product.query.filter_by(company_id=company_id).all()

        for product in products:

            # Step 2: Get inventory across all warehouses
```

```
inventories = Inventory.query.filter_by(product_id=product.id).all()

for inv in inventories:

    # Step 3: Calculate recent sales
    total_sales = db.session.query(func.sum(Sales.quantity_sold))\
        .filter(
            Sales.product_id == product.id,
            Sales.warehouse_id == inv.warehouse_id,
            Sales.sale_date >= recent_date
        ).scalar() or 0

    # Skip if no recent sales
    if total_sales == 0:
        continue

    # Step 4: Calculate average daily sales
    avg_daily_sales = total_sales / recent_days

    # Avoid division by zero
    if avg_daily_sales == 0:
        continue

    # Step 5: Get threshold
    threshold = product.low_stock_threshold or 10

    # Step 6: Check low stock condition
    if inv.quantity < threshold:

        # Step 7: Calculate days until stockout
        days_until_stockout = int(inv.quantity / avg_daily_sales)
```

```
        # Step 8: Get warehouse details
        warehouse = Warehouse.query.get(inv.warehouse_id)

        # Step 9: Get supplier
        supplier = db.session.query(Supplier)\
            .join(Product_Suppliers, Supplier.id == Product_Suppliers.supplier_id)\
            .filter(Product_Suppliers.product_id == product.id)\
            .first()

        alerts.append({
            "product_id": product.id,
            "product_name": product.name,
            "sku": product.sku,
            "warehouse_id": warehouse.id if warehouse else None,
            "warehouse_name": warehouse.name if warehouse else None,
            "current_stock": inv.quantity,
            "threshold": threshold,
            "days_until_stockout": days_until_stockout,
            "supplier": {
                "id": supplier.id if supplier else None,
                "name": supplier.name if supplier else None,
                "contact_email": supplier.contact_email if supplier else None
            }
        })

    return jsonify({
        "alerts": alerts,
        "total_alerts": len(alerts)
    }), 200
```

```
except Exception as e:
    return jsonify({"error": str(e)}), 500
```

---

4. Edge Case Handling

1. **No Products Found**
   → Return empty alerts list
2. **No Recent Sales**
   → Product is ignored (as per business rule)
3. **Zero Average Sales**
   → Avoid division by zero
4. **Missing Supplier**
   → Return supplier fields as null
5. **Invalid Company ID**
   → Return empty result or error response
6. **Multiple Warehouses**
   → Each warehouse evaluated separately

---

5. Explanation of Approach

1. **Company-Level Filtering**
   First fetch all products belonging to the company.
2. **Warehouse-Level Inventory Check**
   Inventory is evaluated per warehouse to handle distributed stock.
3. **Recent Sales Filtering**
   Only products with sales in the last 30 days are considered.
4. **Dynamic Stock Analysis**
   Average daily sales are calculated to estimate stock consumption.
5. **Low Stock Detection**
   Compare current stock with threshold to identify risk.
6. **Stockout Prediction**
   Estimate how many days inventory will last.
7. **Supplier Integration**
   Supplier details are included to support reordering decisions.

6. Sample Response


```
{
  "alerts": [
    {
      "product_id": 101,
      "product_name": "Widget A",
      "sku": "WID-001",
      "warehouse_id": 1,
      "warehouse_name": "Main Warehouse",
      "current_stock": 5,
      "threshold": 20,
      "days_until_stockout": 12,
      "supplier": {
        "id": 10,
        "name": "ABC Suppliers",
        "contact_email": "orders@abc.com"
      }
    }
  ],
  "total_alerts": 1
}
```

This API:

- Identifies **low stock products**
- Considers **real-time sales trends**
- Supports **multi-warehouse systems**
- Provides **actionable supplier insights**
- Is designed for **scalability and production use**